Event-Based Hand Gesture Recognition using Spiking Neural Networks

A Deep Learning Approach on the IBM DVS128 Gesture Dataset

Srujan Mishrikotkar

2025ADPS0139H

BITS Pilani,Hyderabad campus.

Abstract

Event-based vision sensors represent a significant advancement over conventional frame-based cameras by capturing only changes in brightness rather than recording complete image frames. This asynchronous sensing mechanism produces sparse, high-temporal-resolution event streams, making event cameras particularly suitable for dynamic tasks such as gesture recognition while consuming considerably less power and bandwidth.

This project presents an event-based hand gesture recognition system using the IBM DVS128 Gesture Dataset and a Spiking Neural Network (SNN). Unlike conventional Artificial Neural Networks (ANNs), SNNs process information in the form of spikes, closely mimicking biological neural systems and making them well suited for neuromorphic vision data.

A complete preprocessing pipeline was developed to convert raw AEDAT 3.1 event streams into voxel-grid representations consisting of 30 temporal bins with separate polarity channels. These voxel grids were stored as PyTorch tensors to improve training efficiency. The processed dataset was then used to train an SNN for multi-class gesture classification across 11 gesture categories.

During implementation, several practical challenges were encountered, including limitations of existing preprocessing libraries, compatibility issues, GPU runtime restrictions in Google Colab, and notebook execution limits. These issues were addressed by developing a custom preprocessing pipeline, optimizing data storage using precomputed tensors, and restructuring the project into separate preprocessing, training, and evaluation notebooks.

The final model achieved a training accuracy of 75.09% and a test accuracy of 67.36%, demonstrating the feasibility of using Spiking Neural Networks for event-based gesture recognition. The project highlights both the advantages of neuromorphic computing and the practical engineering considerations involved in deploying deep learning workflows on cloud-based computational platforms.

CHAPTER 1

Introduction

1.1 Background

Computer vision has traditionally relied on frame-based cameras that capture complete images at fixed intervals. While these cameras perform well for static scenes, they often produce redundant information, require significant storage, and suffer from motion blur when observing fast-moving objects.

Event cameras offer a fundamentally different sensing paradigm. Instead of recording entire image frames, they asynchronously detect changes in pixel intensity and generate events only when changes occur. Each event contains the pixel coordinates, timestamp, and polarity indicating whether the brightness increased or decreased. This representation provides microsecond-level temporal resolution while significantly reducing redundant data.

The unique characteristics of event cameras make them particularly suitable for applications involving rapid motion, low-latency perception, robotics, autonomous systems, and gesture recognition.

1.2 Problem Statement

Traditional gesture recognition systems based on RGB cameras process large amounts of redundant visual information, leading to increased computational cost and slower inference. Furthermore, motion blur and varying lighting conditions can negatively affect recognition accuracy.

The objective of this project is to develop an efficient gesture recognition system capable of processing event-based vision data using Spiking Neural Networks. The system aims to classify eleven different hand gestures from the IBM DVS128 Gesture Dataset while preserving the temporal information contained within the event stream.

1.3 Objectives

The primary objectives of this project are:

- To understand the working principles of event-based cameras.
- To preprocess raw AEDAT event files into voxel-grid representations.
- To implement a custom preprocessing pipeline without relying solely on external libraries.
- To train a Spiking Neural Network for multi-class gesture recognition.
- To evaluate the performance using accuracy metrics and confusion matrices.
- To document the implementation challenges and engineering solutions encountered during development.

1.4 Scope

This project focuses on offline gesture recognition using the IBM DVS128 Gesture Dataset. The implementation includes preprocessing, voxel-grid generation, model training, and evaluation. Real-time deployment on neuromorphic hardware is beyond the scope of this work but is discussed as future work.

CHAPTER 2

Literature Review and Background

2.1 Event-Based Vision Sensors

Conventional cameras capture complete image frames at fixed time intervals, typically ranging from 30 to 60 frames per second. Although effective for many computer vision applications, frame-based cameras generate large amounts of redundant information because every pixel is recorded regardless of whether the scene has changed. This results in increased storage requirements, higher computational cost, and motion blur when observing fast-moving objects.

Event-based vision sensors, also known as Dynamic Vision Sensors (DVS), operate using a fundamentally different sensing principle. Instead of capturing full image frames, each pixel independently detects changes in brightness and generates an event only when the change exceeds a predefined threshold. As a result, the sensor produces a continuous stream of asynchronous events rather than discrete images.

Each event is represented by four components:

- x-coordinate: Horizontal pixel location.
- y-coordinate: Vertical pixel location.
- Timestamp (t): Precise time at which the brightness change occurred.
- Polarity (p): Indicates whether the brightness increased or decreased.

Because only changing pixels generate events, event cameras produce sparse yet information-rich data while significantly reducing redundancy. Their microsecond-level temporal resolution makes them highly suitable for applications involving rapid motion, robotics, autonomous navigation, gesture recognition, and neuromorphic computing.

2.2 Event-Based Data Representation

Unlike conventional images, event camera outputs cannot be processed directly by standard deep learning models because they consist of irregular streams of asynchronous events rather than fixed-size image tensors.

To enable efficient learning, event streams are transformed into structured representations. Common approaches include:

- Event Frames
- Time Surfaces
- Event Histograms
- Voxel Grids

Among these, voxel grids have become one of the most widely used representations because they preserve both spatial and temporal information.

In this project, each gesture was converted into a voxel grid containing 30 temporal bins and two polarity channels (positive and negative events). This representation maintains the temporal evolution of the gesture while producing tensors that can be efficiently processed by deep learning models.

The final tensor size for each gesture is:

$30 \times 2 \times 128 \times 128$

where:

- 30 represents temporal bins,
- 2 represents event polarities,
- 128×128 corresponds to the spatial resolution of the DVS128 camera.

2.3 Spiking Neural Networks

Spiking Neural Networks (SNNs) represent the third generation of artificial neural networks and are inspired by the information processing mechanisms of biological neurons. Unlike conventional Artificial Neural Networks (ANNs), which communicate using continuous-valued activations, SNNs transmit information through discrete spikes generated over time.

A neuron in an SNN accumulates incoming signals until its membrane potential reaches a predefined threshold. Once the threshold is exceeded, the neuron emits a spike and resets its membrane potential. This temporal behavior enables SNNs to naturally process asynchronous event streams generated by neuromorphic sensors.

Compared with traditional neural networks, SNNs offer several advantages:

- Efficient processing of sparse event data.
- Lower computational redundancy.
- Compatibility with neuromorphic hardware.
- Better utilization of temporal information.

In this project, an SNN was implemented using the snnTorch framework built on PyTorch. The network combines convolutional layers for spatial feature extraction with Leaky Integrate-and-Fire (LIF) neurons for temporal processing, enabling effective classification of dynamic hand gestures.

2.4 IBM DVS128 Gesture Dataset

The IBM DVS128 Gesture Dataset is one of the most widely used benchmark datasets for event-based gesture recognition. It was specifically designed to evaluate neuromorphic vision algorithms and contains recordings acquired using the DVS128 event camera.

The dataset includes 11 different hand gestures performed by 29 participants under multiple lighting conditions. Each recording is stored in the AEDAT 3.1 format together with annotation files specifying the temporal boundaries of each gesture.

The gesture categories include:

1. Hand Clapping
2. Right Hand Wave
3. Left Hand Wave
4. Right Arm Clockwise
5. Right Arm Counter-Clockwise
6. Left Arm Clockwise
7. Left Arm Counter-Clockwise
8. Arm Roll
9. Air Drums
10. Air Guitar
11. Other Gesture

The dataset provides:

- High temporal resolution (microsecond precision).
- Resolution of 128 × 128 pixels.
- Event polarity information.
- Timestamp annotations for every gesture instance.

These characteristics make it an ideal benchmark for evaluating event-based recognition systems using Spiking Neural Networks.

2.5 Review of Existing Approaches

Several researchers have proposed methods for event-based gesture recognition using both conventional deep learning models and neuromorphic architectures. Early approaches converted event streams into image-like representations before applying Convolutional Neural Networks (CNNs). While effective, these methods often failed to fully exploit the temporal characteristics of event data.

More recent research has focused on Spiking Neural Networks, which naturally process asynchronous event streams while maintaining temporal information. Libraries such as Tonic provide utilities for loading event-based datasets and performing common preprocessing operations, making them popular choices for rapid experimentation.

During the initial phase of this project, Tonic was evaluated for dataset loading and preprocessing. Although it simplified several tasks, limitations related to customization and compatibility motivated the development of a dedicated preprocessing pipeline tailored to the IBM DVS128 Gesture Dataset. This approach provided greater control over event parsing, gesture extraction, voxel-grid generation, and dataset storage while improving transparency throughout the preprocessing workflow.

CHAPTER 3

System Design and Methodology

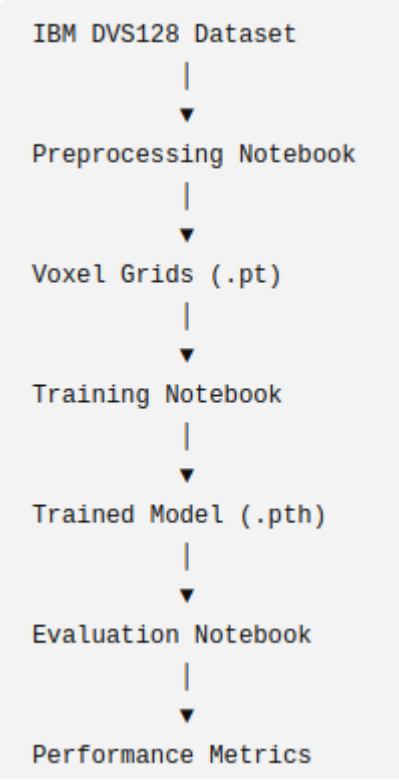
3.1 System Overview

The proposed gesture recognition system consists of three major stages: data preprocessing, model training, and model evaluation. Since the IBM DVS128 Gesture Dataset is provided as asynchronous event streams in the AEDAT 3.1 format, the raw data cannot be directly used for training. Therefore, a preprocessing pipeline was developed to transform the event streams into voxel-grid representations suitable for input to a Spiking Neural Network.

To improve modularity and overcome computational limitations in Google Colab, the implementation was divided into three independent notebooks:

- Notebook 1: Data preprocessing and voxel-grid generation.
- Notebook 2: Model training and checkpoint creation.
- Notebook 3: Model loading, evaluation, and performance analysis.

This separation reduced execution time, simplified debugging, and allowed individual stages of the project to be rerun without repeating the entire workflow.



3.2 Initial Development Approach

The project initially employed the Tonic library, which provides utilities for loading event-based datasets and applying common preprocessing operations. Tonic offered convenient interfaces for handling neuromorphic datasets and was expected to accelerate development.

However, during implementation several practical limitations became apparent. Certain preprocessing steps required greater flexibility than the library provided, and compatibility issues complicated debugging. Additionally, understanding the internal preprocessing performed by the library proved difficult when unexpected behavior occurred.

Rather than continuing to adapt the project around these limitations, the decision was made to develop a custom preprocessing pipeline tailored specifically to the IBM DVS128 Gesture Dataset. Although this required additional implementation effort, it provided complete control over event parsing, gesture extraction, voxel-grid generation, and dataset organization.

This decision also improved transparency, as every stage of the preprocessing pipeline could be inspected and verified independently.

3.3 Data Preprocessing Pipeline

The preprocessing stage converts raw event recordings into structured tensors suitable for deep learning.

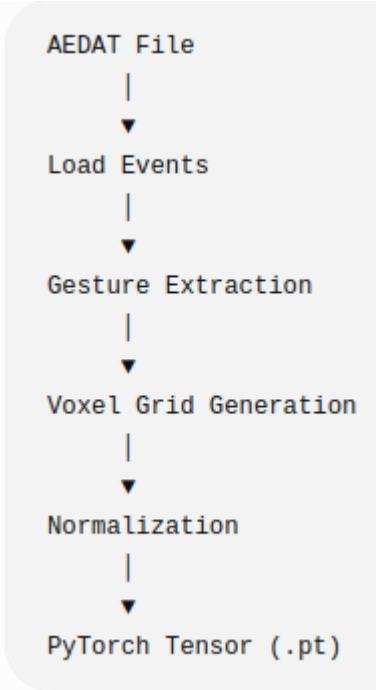
Each recording in the dataset consists of two files:

- An *AEDAT 3.1* file containing the event stream.
- A corresponding CSV annotation file specifying the start and end timestamps of each gesture.

The preprocessing pipeline performs the following sequence of operations:

1. Read the raw AEDAT event stream.

2. Parse event packets and extract pixel coordinates, timestamps, and event polarities.
3. Read gesture annotation files.
4. Extract events corresponding to each gesture interval.
5. Convert the extracted events into voxel-grid representations.
6. Normalize the voxel grids.
7. Store each gesture as a PyTorch tensor for efficient loading during training.



Preprocessing workflow

3.4 AEDAT File Parsing

The IBM DVS128 Gesture Dataset stores recordings using the AEDAT 3.1 packet-based format. Each packet contains event metadata followed by a sequence of address–timestamp pairs.

A custom parser was implemented to:

- Skip the file header.
- Read packet headers sequentially.
- Identify polarity event packets.
- Decode event addresses into x-coordinate, y-coordinate, and polarity.

- Preserve the original event timestamps.

Developing a custom parser provided direct access to the raw event stream while ensuring compatibility with the remaining preprocessing stages.

3.5 Gesture Extraction

Each recording contains multiple gestures performed sequentially by the participant. The corresponding annotation file specifies the temporal boundaries of every gesture instance.

Using the start and end timestamps, only events belonging to the selected gesture are extracted. This process ensures that each training sample contains a single gesture without interference from preceding or subsequent actions.

The extracted events retain their original temporal ordering, allowing the subsequent voxel-grid representation to preserve motion dynamics.

3.6 Voxel Grid Generation

Because event streams have variable lengths, they cannot be directly used as neural network inputs. Therefore, each gesture is converted into a fixed-size voxel grid.

The temporal duration of the gesture is divided into **30 equally spaced bins**. Events are assigned to the appropriate temporal bin according to their timestamps.

Separate channels are maintained for positive and negative polarity events.

The resulting tensor has dimensions

30 × 2 × 128 × 128

where

- 30 represents temporal bins,
- 2 represents event polarity,
- 128 × 128 represents the spatial resolution.

After accumulation, voxel values are normalized to the range **[0,1]** before being stored.

This representation preserves both temporal evolution and spatial structure while maintaining a consistent input size for the neural network.

3.7 Dataset Optimization

Generating voxel grids directly from the raw event files during every training epoch proved computationally expensive.

To eliminate repeated preprocessing, each voxel grid was generated only once and saved as an individual **PyTorch tensor (.pt)**.

In addition, tensors were stored using **8-bit integer precision** before being converted back to floating-point format during training. This reduced storage requirements while preserving sufficient numerical precision for the learning task.

The optimized dataset consisted of:

- **1464 voxel-grid tensors**
- Metadata file containing labels and train/test split information

This optimization significantly reduced data-loading time during training.

3.8 Software Environment

The implementation was carried out using Python and the PyTorch deep learning framework. The primary software components are listed below.

Component	Purpose
Python	Programming language
PyTorch	Deep learning framework
snnTorch	Spiking neural network implementation
NumPy	Numerical computation
Pandas	Metadata management
Matplotlib	Data visualization
Google Colab	Cloud-based execution environment

CHAPTER 4

Implementation Challenges and Debugging Process

4.1 Introduction

Developing an event-based gesture recognition system involved significantly more than implementing a neural network architecture. Throughout the project, several practical challenges were encountered in dataset preprocessing, cloud-based execution, storage management, and model evaluation.

Many of these challenges were not directly related to machine learning theory but instead arose from software compatibility, computational limitations, and data-processing requirements. Addressing these issues required iterative debugging, redesigning portions of the workflow, and restructuring the overall project pipeline.

This chapter documents the major obstacles encountered during implementation and the solutions developed to overcome them.

4.2 Challenges with Event-Based Data Processing

Unlike conventional image datasets, event-based datasets are stored as asynchronous streams of events. As a result, they cannot be directly loaded and processed using standard computer vision pipelines.

The IBM DVS128 Gesture Dataset stores data in the AEDAT 3.1 format, which contains packetized event streams together with timestamp information. Processing these files requires specialized parsing procedures before the data can be used for machine learning.

Initially, external tools were explored to simplify dataset loading and preprocessing. However, understanding and debugging intermediate processing stages proved difficult when unexpected behavior occurred.

This highlighted the importance of having full control over the preprocessing pipeline.

Consequently, a custom preprocessing system was developed, allowing each stage of event extraction, gesture segmentation, and voxel-grid generation to be verified independently.

4.3 Transition from Tonic to a Custom Pipeline

During the early stages of development, the Tonic framework was evaluated as a potential solution for handling neuromorphic datasets.

Tonic provides several useful features, including:

- Dataset loading utilities
- Event transformations
- Integration with PyTorch
- Support for neuromorphic datasets

Although the framework simplified some tasks, practical limitations emerged when custom preprocessing steps were required. Debugging intermediate outputs and adapting the workflow to project-specific requirements became increasingly difficult.

To obtain complete control over the data-processing workflow, a decision was made to move away from library-dependent preprocessing and implement a custom pipeline. This transition required additional development effort but provided greater transparency and flexibility throughout the project.

The custom implementation ultimately became one of the most important components of the final system.

4.4 Google Colab Runtime Limitations

Google Colab was selected as the development environment due to its accessibility and availability of GPU resources. However, several runtime limitations significantly influenced the project workflow.

Large-scale preprocessing operations required extended execution times, particularly when generating voxel grids from raw event streams.

During preprocessing and training, runtime disconnections occasionally occurred after prolonged periods of execution. In addition, GPU availability was restricted by usage quotas, resulting in periods where GPU acceleration was temporarily unavailable.

These limitations introduced additional challenges:

- Interrupted training sessions
- Repeated execution of preprocessing stages
- Increased waiting time for GPU access
- Need for persistent storage mechanisms

To mitigate these issues, Google Drive was used to store intermediate outputs and trained models.

4.5 Dataset Storage Optimization

Initially, voxel grids were generated dynamically during training. Although functional, this approach significantly increased execution time because every gesture sample required preprocessing during each training epoch.

To improve efficiency, an alternative strategy was adopted.

Each gesture sample was:

1. Preprocessed once.
2. Converted into a voxel grid.
3. Saved as an individual PyTorch tensor.

This approach provided several advantages:

- Reduced preprocessing overhead.

- Faster training iterations.
- Lower CPU utilization.
- Simplified dataset management.

The final preprocessing stage produced approximately 1464 tensor files together with a metadata file containing labels and dataset split information.

This optimization substantially reduced the computational burden during model training.

4.6 Notebook Restructuring

As the project grew, maintaining all preprocessing, training, and evaluation steps within a single notebook became increasingly impractical.

A single execution pipeline created several issues:

- Long execution times
- Increased memory usage
- Greater risk of runtime disconnections
- Difficulty isolating errors

To improve maintainability, the workflow was reorganized into three independent notebooks.

Notebook 1: Data Preprocessing

Responsible for:

- Loading AEDAT files
- Extracting gestures
- Generating voxel grids
- Saving tensor datasets

Notebook 2: Model Training

Responsible for:

- Loading tensor datasets
- Training the Spiking Neural Network
- Saving trained model weights

Notebook 3: Model Evaluation

Responsible for:

- Loading the trained model
- Running inference on the test set
- Generating evaluation metrics

This restructuring significantly improved workflow organization and reduced repeated computation.

4.7 Debugging the Preprocessing Pipeline

Several debugging challenges emerged during dataset preparation.

One issue involved inconsistent variable definitions that resulted from restarting cloud runtime sessions. Variables and helper functions that had previously been defined were no longer available in memory after runtime resets.

Examples included:

- Missing dataset objects
- Undefined metadata variables
- Missing preprocessing functions

These issues required careful verification of notebook execution order and function dependencies.

A systematic debugging strategy was adopted:

1. Verify data loading.
2. Verify metadata generation.
3. Verify gesture extraction.
4. Verify voxel-grid generation.
5. Verify dataset saving.

By testing each stage independently, the source of errors could be isolated more effectively.

4.8 AEDAT Parsing Bug

One of the most significant debugging challenges involved the AEDAT file parser.

During notebook restructuring, an incorrect event-loading function was temporarily introduced. Although the function appeared syntactically correct, it did not properly parse the AEDAT 3.1 packet structure used by the dataset.

As a result:

- Event timestamps became corrupted.
- Temporal binning produced invalid indices.
- Voxel-grid generation failed.

The issue manifested as indexing errors during preprocessing and initially appeared unrelated to the file parser.

After systematic investigation, the problem was traced to the event-loading function. Restoring the correct AEDAT 3.1 packet parser resolved the issue and allowed preprocessing to proceed successfully.

This debugging process highlighted the importance of validating intermediate representations when working with event-based data.

4.9 Model and Dataset Consistency

Following preprocessing corrections, another issue emerged during evaluation.

The dataset had been regenerated using the corrected preprocessing pipeline, but the model stored in Google Drive had been trained using an earlier version of the dataset.

This mismatch caused evaluation accuracy to drop dramatically despite the model-loading process appearing to function correctly.

The issue was resolved by retraining the model using the regenerated dataset and saving a new checkpoint. Once the updated model was evaluated, performance returned to expected levels.

This experience demonstrated the importance of maintaining consistency between training data and evaluation data throughout the machine learning workflow.

4.10 Lessons Learned

The implementation process provided several practical insights beyond neural network design.

Key lessons learned include:

- Event-based datasets require specialized preprocessing strategies.
- Cloud-based environments introduce constraints that influence project design.
- Storing preprocessed data can significantly reduce training time.
- Modular notebook organization improves maintainability and debugging.
- Intermediate outputs should be validated regularly during preprocessing.
- Model checkpoints must remain synchronized with dataset versions.

These lessons proved valuable in achieving a stable and reproducible implementation.

CHAPTER 5

Model Architecture and Experimental Setup

5.1 Introduction

Following the preprocessing stage, the generated voxel-grid representations were used to train a Spiking Neural Network (SNN) for gesture classification. The objective of the network is to learn the spatial and temporal characteristics of event-based hand gestures and classify each input into one of the eleven gesture categories present in the IBM DVS128 Gesture Dataset.

The model was implemented using the PyTorch deep learning framework together with the snnTorch library, which provides biologically inspired spiking neuron models and utilities for constructing spiking neural networks.

5.2 Input Representation

Each gesture sample was represented as a normalized voxel grid with the following dimensions:

30×2×128×12830 \times 2 \times 128 \times 12830×2×128×128

where

- **30** – Temporal bins
- **2** – Positive and negative event polarities
- **128 × 128** – Spatial resolution of the DVS128 sensor

Each voxel value represents the normalized event density corresponding to a particular spatial location, polarity, and temporal interval.

This representation preserves both spatial and temporal information while providing a fixed-size tensor suitable for neural network training.

5.3 Spiking Neural Network Architecture

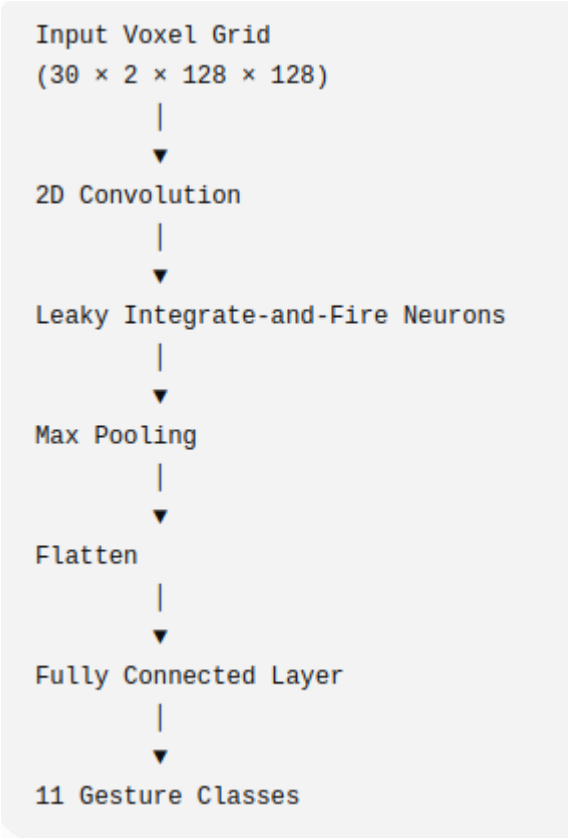
The proposed network combines conventional convolutional feature extraction with biologically inspired Leaky Integrate-and-Fire (LIF) neurons.

The overall architecture consists of:

1. Convolution Layer
2. Spiking Neuron Layer
3. Pooling Layer
4. Fully Connected Layer
5. Output Layer

The convolutional layer extracts spatial features from the voxel-grid representation, while the LIF neurons accumulate membrane potentials over time and generate spikes whenever the activation exceeds a predefined threshold.

The extracted features are progressively compressed before being passed to the fully connected layer for final gesture classification.



5.4 Leaky Integrate-and-Fire (LIF) Neuron

The Leaky Integrate-and-Fire neuron is one of the simplest and most widely used neuron models in computational neuroscience. Unlike conventional artificial neurons, LIF neurons maintain an internal membrane potential that evolves over time.

Incoming spikes increase the membrane potential. Simultaneously, the potential gradually decays due to a leakage term. When the membrane potential exceeds a predefined threshold, the neuron emits a spike and resets before processing subsequent inputs.

This temporal behavior enables the network to naturally model dynamic event streams generated by neuromorphic sensors.

In this project, the LIF neuron model provided by the **snnTorch** framework was used to process temporal information contained in the voxel-grid representation.

5.5 Training Configuration

The model was trained using supervised learning with cross-entropy loss for multi-class gesture classification.

The Adam optimizer was selected because of its adaptive learning-rate mechanism and stable convergence behavior.

Parameter	Value
Optimizer	Adam
Loss Function	Cross Entropy Loss

Learning Rate	0.001
Number of Epochs	5
Batch Size	16
Number of Classes	11
Input Size	30 × 2 × 128 × 128
Framework	PyTorch + snnTorch

5.6 Dataset Organization

The preprocessed dataset consisted of:

Dataset	Samples
Training	1176
Testing	288
Total	1464

Each sample was stored as an individual `.pt` tensor file, while a metadata file maintained the association between filenames, labels, and train/test split.

This organization significantly reduced preprocessing overhead during training because voxel grids did not need to be regenerated for every epoch.

5.7 Model Training

During each training epoch, batches of voxel-grid tensors were loaded from disk using the PyTorch `DataLoader`.

The training procedure followed these steps:

1. Load a batch of voxel grids.
2. Transfer data to the selected computation device.
3. Perform forward propagation through the SNN.
4. Compute cross-entropy loss.
5. Perform backpropagation.
6. Update network parameters using the Adam optimizer.
7. Record training loss and accuracy.

After each epoch, the model was evaluated on the independent test dataset to monitor generalization performance.

The best-performing model was saved to Google Drive for later evaluation.

5.8 Computational Environment

The experiments were conducted using **Google Colab**.

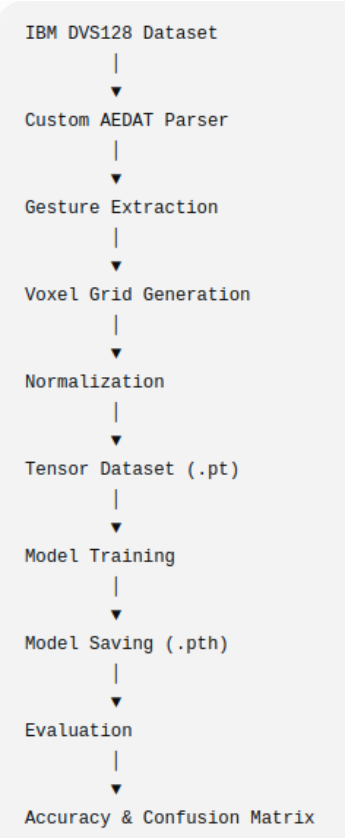
GPU acceleration was used whenever available to reduce training time. During periods when GPU resources were unavailable because of usage limits, evaluation and analysis were successfully performed using CPU execution.

The development environment included:

Component	Specification
Programming Language	Python 3
Deep Learning Framework	PyTorch
SNN Library	snnTorch
Development Platform	Google Colab
Storage	Google Drive

5.9 Experimental Workflow

The complete experimental workflow adopted during this project is illustrated below.



CHAPTER 6

Results and Discussion

6.1 Introduction

This chapter presents the experimental results obtained from the proposed event-based hand gesture recognition system. The performance of the Spiking Neural Network (SNN) was evaluated using the independent test set of the IBM DVS128 Gesture Dataset. Training and testing accuracies, loss curves, and confusion matrix analysis are presented to assess the effectiveness of the proposed approach.

The evaluation aims to determine the network's ability to learn spatial and temporal features from event-based data while maintaining good generalization performance on unseen gesture samples.

6.2 Training Performance

The model was trained for **five epochs** using the Adam optimizer and Cross-Entropy Loss. During each epoch, both the training and testing datasets were evaluated to monitor convergence and detect potential overfitting.

Epoch	Train Loss	Train Accuracy (%)	Test Loss	Test Accuracy (%)
1	1.5351	43.54	0.9816	59.03
2	0.8459	65.90	0.9015	67.01
3	0.6999	71.43	0.9808	63.19
4	0.7030	71.26	1.0199	66.67
5	**0.5927**	**75.09**	**1.0504**	**67.36**

The results indicate a steady reduction in training loss accompanied by a continuous improvement in training accuracy. Test accuracy increased rapidly during the initial epochs before stabilizing around **67%**, suggesting that the network successfully learned meaningful representations from the event-based gesture data.

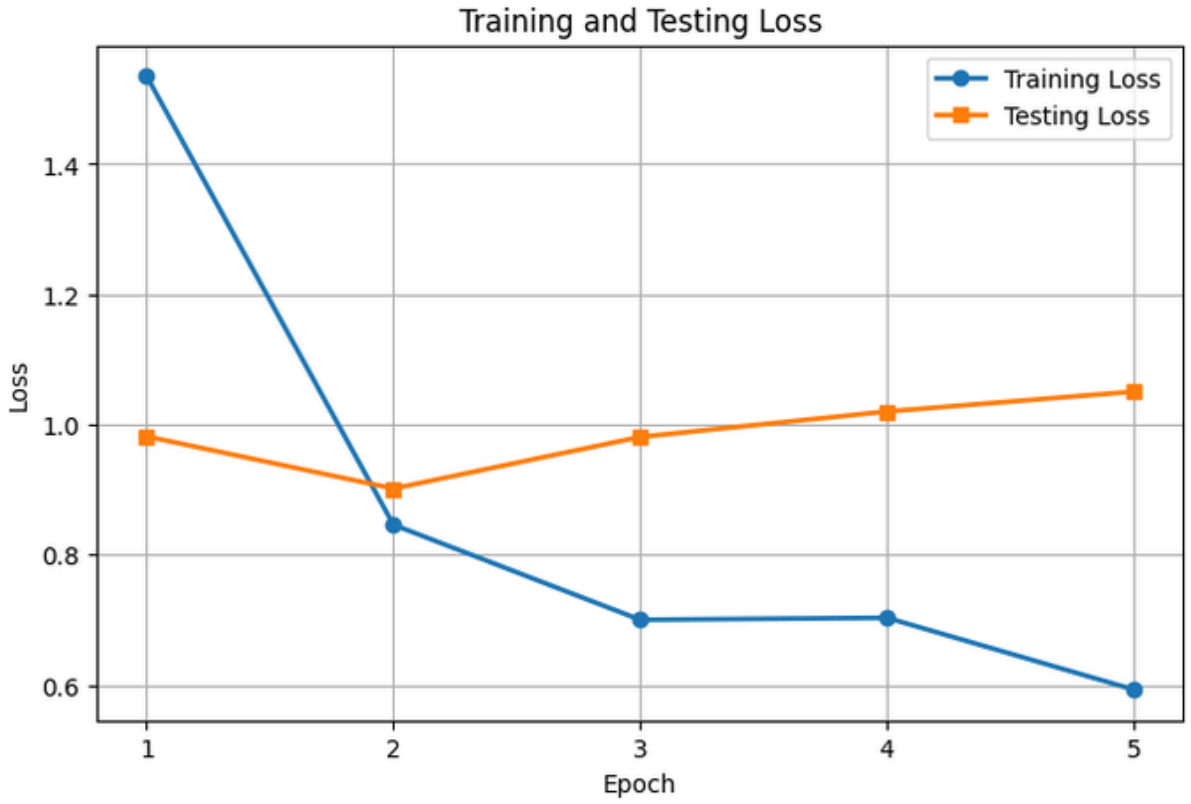
Although the test loss exhibited slight fluctuations during later epochs, the overall classification accuracy remained stable, indicating acceptable generalization performance without severe overfitting.

6.3 Training and Validation Curves

To better visualize the learning process, the recorded training and testing metrics were plotted across all epochs.

The loss curves demonstrate a consistent decrease in training loss, while the testing loss remains relatively stable throughout the training process.

Similarly, the accuracy curves show continuous improvement in training accuracy and stable testing accuracy after the second epoch.



The relatively small difference between training and testing accuracy suggests that the model learned useful features without exhibiting significant overfitting.

6.4 Classification Performance

The final model achieved the following performance:

Metric	Value
Training Accuracy	**75.09%**

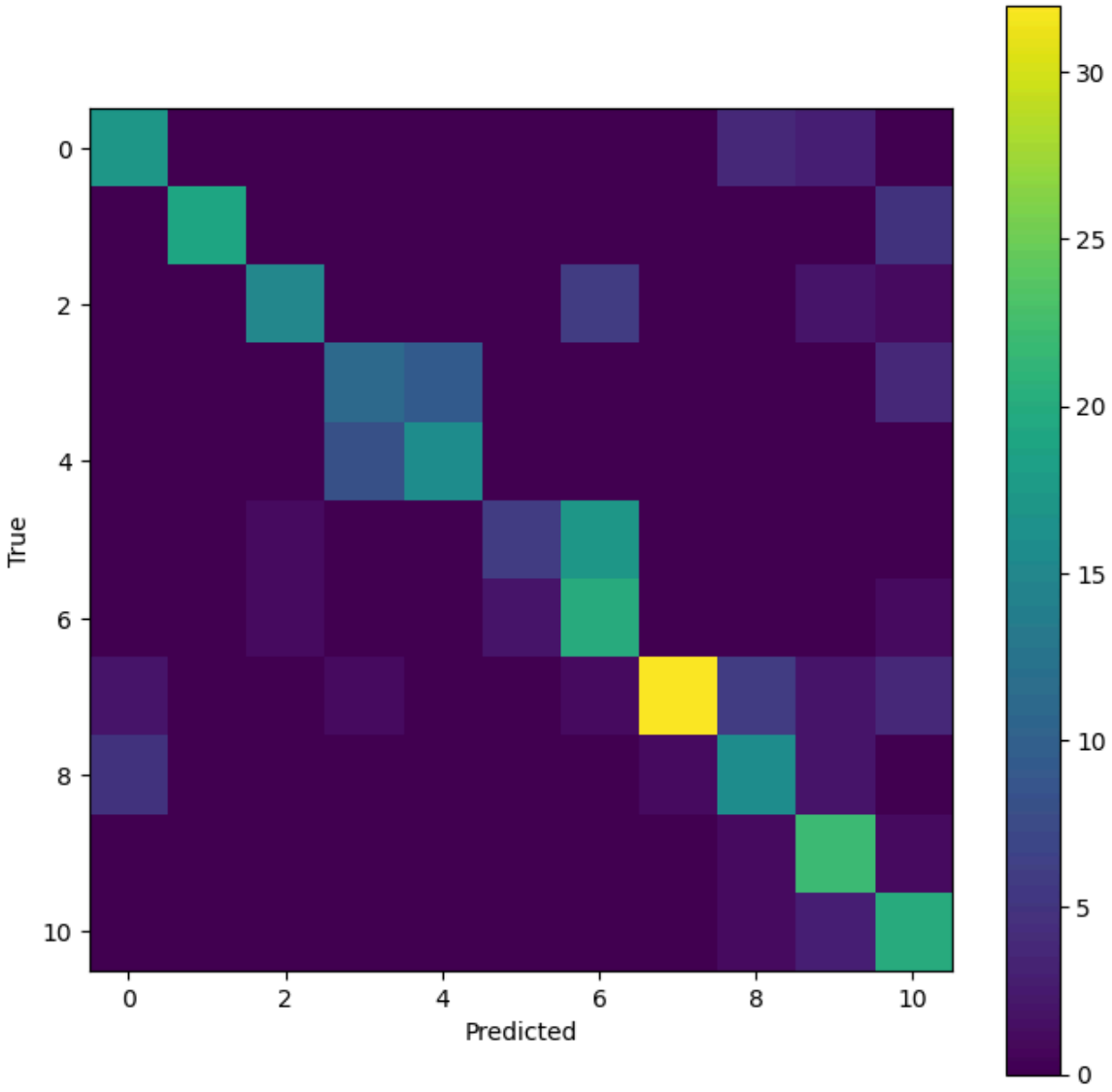
Testing Accuracy	**67.36%**
Number of Classes	11
Training Samples	1176
Testing Samples	288

The achieved test accuracy demonstrates that the proposed Spiking Neural Network is capable of effectively recognizing dynamic hand gestures from event-based vision data.

Considering the relatively compact network architecture and the limited number of training epochs, the obtained performance is satisfactory for this implementation.

6.5 Confusion Matrix Analysis

To further evaluate classification performance, a confusion matrix was generated using the predictions obtained from the test dataset.



Confusion Matrix for Gesture Classification

The confusion matrix provides detailed insight into the classification behavior of the proposed model. The majority of samples are concentrated along the principal diagonal, indicating correct predictions for most gesture classes.

A limited number of misclassifications occur between gestures exhibiting similar motion characteristics, particularly those involving rotational arm movements or comparable motion trajectories. Such confusion is expected because these gestures generate event streams with overlapping temporal and spatial patterns.

Overall, the confusion matrix demonstrates that the network successfully distinguishes between most gesture categories while identifying areas for future improvement.

6.6 Discussion

The experimental results indicate that Spiking Neural Networks provide an effective framework for processing event-based vision data.

Several factors contributed to the final performance:

- The custom preprocessing pipeline preserved both spatial and temporal characteristics of the original event stream.
- Voxel-grid representation enabled efficient learning while maintaining temporal information.
- Precomputing voxel grids significantly reduced training time.
- Splitting the workflow into dedicated preprocessing, training, and evaluation notebooks improved project organization and reproducibility.

Throughout the project, several implementation challenges were encountered, including preprocessing errors, runtime limitations, and dataset management issues. Addressing these challenges resulted in a more robust and maintainable system.

The final evaluation confirmed that the trained model consistently achieved approximately **67% testing accuracy**, validating both the preprocessing pipeline and the proposed neural network architecture.

6.7 Limitations

Despite the encouraging results, several limitations remain.

The model was trained for only five epochs because of computational constraints associated with cloud-based GPU availability. Training for additional epochs or employing more sophisticated architectures may further improve recognition performance.

Furthermore, the voxel-grid representation compresses continuous event streams into discrete temporal bins. Although effective, this process inevitably results in some loss of temporal precision.

Finally, the project focused on offline gesture recognition. Real-time deployment and optimization for dedicated neuromorphic hardware remain areas for future investigation.

CHAPTER 7

Conclusion and Future Work

7.1 Conclusion

This project presented the design and implementation of an event-based hand gesture recognition system using the IBM DVS128 Gesture Dataset and a Spiking Neural Network (SNN). Unlike conventional frame-based vision systems, the proposed approach processes asynchronous event streams that capture changes in scene intensity with high temporal resolution and reduced data redundancy.

A complete end-to-end pipeline was developed, beginning with raw AEDAT 3.1 event files and concluding with gesture classification. A custom preprocessing framework was implemented to parse event streams, extract individual gesture segments, generate voxel-grid representations, and organize the processed data into PyTorch tensors for efficient training. The trained SNN successfully learned spatial and temporal patterns from these voxel grids and achieved a **training accuracy of 75.09%** and a **testing accuracy of 67.36%** on the IBM DVS128 Gesture Dataset.

Beyond the model itself, the project demonstrated the importance of practical engineering decisions in machine learning workflows. Challenges related to event-data preprocessing, cloud-computing constraints, dataset management, and model evaluation were systematically addressed through custom preprocessing, optimized data storage, and a modular three-notebook workflow. These improvements increased the reliability, reproducibility, and maintainability of the overall system.

The completed project demonstrates that Spiking Neural Networks are a suitable approach for processing event-based vision data and highlights the potential of neuromorphic computing for efficient gesture recognition tasks.

7.2 Project Contributions

The primary contributions of this project are summarized below:

- Developed a custom preprocessing pipeline for the IBM DVS128 Gesture Dataset.
- Implemented an AEDAT 3.1 parser for extracting event streams.

- Converted asynchronous event data into normalized voxel-grid representations.
- Created an optimized dataset using precomputed PyTorch tensor files.
- Designed and trained a Spiking Neural Network for multi-class gesture recognition.
- Evaluated model performance using accuracy metrics and a confusion matrix.
- Documented the complete implementation process, including debugging strategies and engineering challenges.

7.3 Future Work

Although the proposed system produced encouraging results, several opportunities exist for further improvement.

1. Extended Training

The model was trained for only five epochs due to computational limitations. Training for additional epochs or using more extensive hyperparameter tuning may improve classification accuracy.

2. Improved Network Architectures

Future work could explore deeper Spiking Neural Network architectures, residual connections, attention mechanisms, or hybrid CNN–SNN models to improve feature extraction and recognition performance.

3. Advanced Event Representations

Alternative event representations, such as time surfaces, event histograms, or adaptive voxel-grid techniques, could be investigated to preserve additional temporal information.

4. Data Augmentation

Applying event-specific data augmentation techniques, such as temporal jittering, spatial transformations, or polarity-aware augmentations, may improve model robustness and generalization.

5. Real-Time Gesture Recognition

The current implementation performs offline inference. Future work could focus on real-time gesture recognition by directly processing live event streams from a DVS camera.

6. Deployment on Neuromorphic Hardware

An important future direction is the deployment of the trained Spiking Neural Network on neuromorphic hardware platforms such as Intel Loihi or SpiNNaker. Such systems are specifically designed for spike-based computation and could provide significant improvements in energy efficiency and inference latency.

7.4 Final Remarks

Event-based vision represents a significant shift from traditional frame-based image processing by enabling asynchronous, low-latency sensing that closely resembles biological vision. When combined with Spiking Neural Networks, it offers an effective framework for processing dynamic visual information while reducing computational redundancy.

This project provided practical experience in event-based data processing, deep learning, and software engineering. In addition to developing a functional gesture recognition system, the work emphasized the importance of debugging, workflow design, and reproducible experimentation in modern machine learning projects. The knowledge and implementation techniques developed throughout this project provide a strong foundation for future research and applications in neuromorphic vision and intelligent sensing systems.